

Final Mastery Report

1. Roughly how many submissions did it take before you passed all critical scenarios, and what was the most common failure?

It took about 3-4 meaningful ChaosArena submissions before all critical scenarios passed reliably and of-course local testing. The most common early failure was in the async photo path, especially upload and completion behavior while the system was moving from local storage to AWS-backed storage.

2. Where are your photo files stored, and why did you pick that over other options?

Final photo files are stored in Amazon S3. I picked S3 because it is durable, scalable under concurrent access, and much better suited for ChaosArena load testing than ECS task-local disk.

3. Describe your deployment setup — how many instances, what cloud services, and how they connect to each other.

The final deployment used 6 ECS Fargate tasks behind an AWS Application Load Balancer in `us-west-2`. The Go API ran in ECS, album and photo metadata lived in DynamoDB, and photo objects were stored in S3; the ALB routed HTTP traffic to ECS, and ECS called DynamoDB and S3 directly.

4. Did you use a reverse proxy or load balancer? If so, what role does it play in your architecture?

Yes, I used an AWS Application Load Balancer. It provided the single public base URL required by the assignment, distributed traffic across ECS tasks, and performed health checks on `/health`.

5. How does your background worker get notified that there's a new photo to process? Did you use a queue, polling, or something else?

Inside the app, the POST handler enqueues the new `photo_id` onto an in-process worker channel immediately after metadata is written. On startup, the service also scans for any photo records still marked `processing` and re-enqueues them, so it combines immediate in-memory signaling with recovery polling.

6. The spec requires that `seq` is assigned in the POST handler, not the background worker. Why does that matter, and how did you ensure correctness under concurrent uploads to the same album?

It matters because the client must get the correct sequence number immediately in the `202 Accepted` response, and the sequence must reflect submission order within an album even before background work finishes. I enforced this with an atomic DynamoDB update on the album record that increments `next_seq` in the POST handler, so concurrent uploads to the same album cannot get duplicate or out-of-order values.

7. What happens in your system if the worker crashes or fails halfway through processing a photo?

If a worker fails during processing, the photo remains in the metadata store as `processing` or is marked `failed` if the error is known. On service startup, any lingering `processing` records are scanned and re-enqueued, so unfinished work can be retried instead of being lost.

8. What does your database schema look like? What tables or collections did you create and why?

I used two DynamoDB tables: one for albums and one for photos. The albums table stores album metadata plus the per-album `next_seq` counter, and the photos table stores `photo_id`, `album_id`, `seq`, `status`, object key, URL, and temporary processing metadata.

9. Did you add any indexes to your database? If so, on which columns and why?

In the final DynamoDB design, I did not need secondary indexes because the core access pattern was direct lookup by primary key. Earlier in the SQLite version I added an index on `photos(album_id)` to make album-photo lookups more efficient.

10. Which load testing scenario was the hardest for you, and what bottleneck did you discover?

`S12` concurrent photo uploads was the hardest. It exposed that my upload path was doing too much work before returning `202`, especially cloud storage transfer and photo completion work under concurrency.

11. What was the single most impactful change you made to improve your load test scores?

The single most impactful change was moving the S3 upload out of the POST request path and into the background worker. That made `202 Accepted` fast and reduced the request-path cost enough to dramatically improve `S12`, `S14`, and `S15`.

12. How did you handle concurrent writes — for example, many album creates or photo uploads happening at the same time?

Album upserts were handled idempotently, and per-album photo sequence assignment used an atomic DynamoDB counter update. That let many clients write concurrently without duplicate albums, duplicate sequence numbers, or race conditions in the upload lifecycle.

13. Describe a specific bug you ran into and how you diagnosed it using the ChaosArena event logs or your own logs.

One concrete bug was that photo uploads in AWS mode returned `500` because S3 rejected the request with `MissingContentLength`. I diagnosed it by adding server-side logging, reading the CloudWatch log line for the failed request, and then changing the code to spool uploads to a temp file so the worker could upload with a known content length.

14. How did you test your service locally before submitting to ChaosArena?

I used `go test ./...` for automated regression coverage and also ran end-to-end smoke tests against the live HTTP service. Those tests covered album create/read/list, async upload, polling to `completed`, fetching the media URL, and delete behavior.

15. If you had another week, what is the one thing you would change or add to your system to improve your score?

If I had another week, I would replace the in-process worker channel with a managed queue such as SQS. That would make the processing pipeline more fault-tolerant across task restarts and would decouple upload acceptance from worker execution even more cleanly.